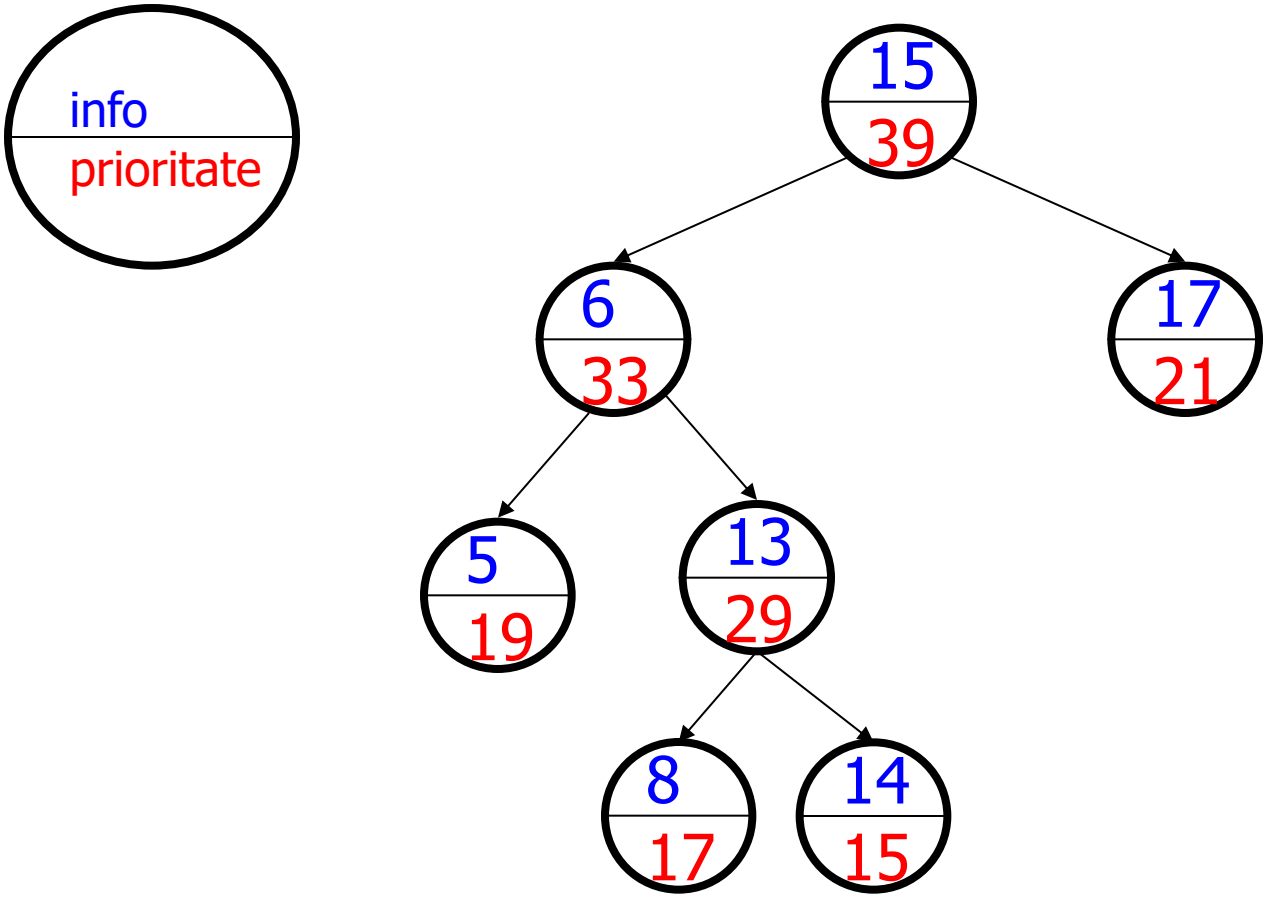
Treap tree

- Tree + Heap → Treap
- Arbore binar ce contine elemente formate din *(info, prioritate)*

- Treap:
  - Arbore binar de cautare (*info*)
  - Heap (*prioritate)*.

Exemplu treap



Operatii

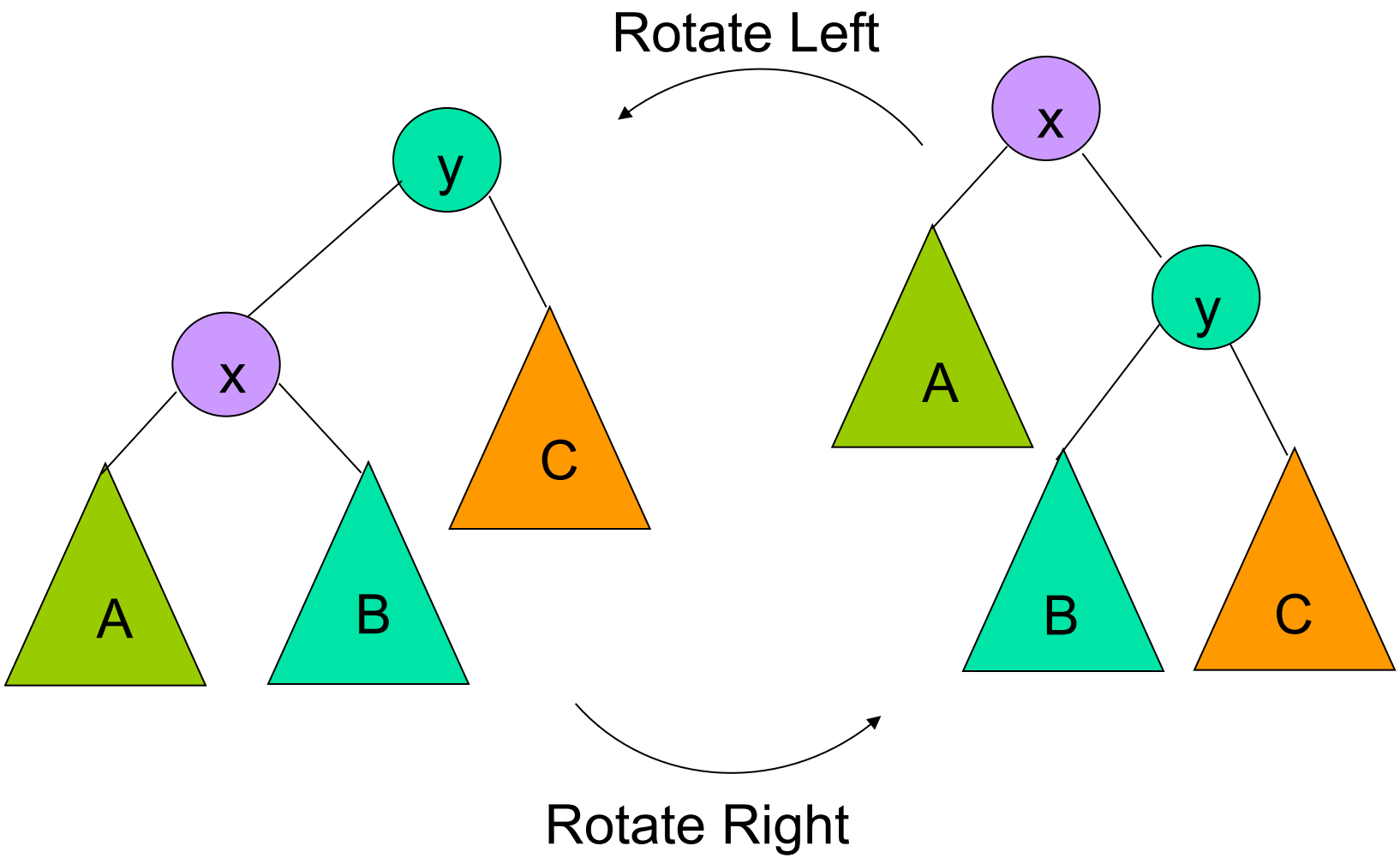
- Cautare – analog arbori binari de cautare
- Inserare
- Eliminare

Inserare in treap

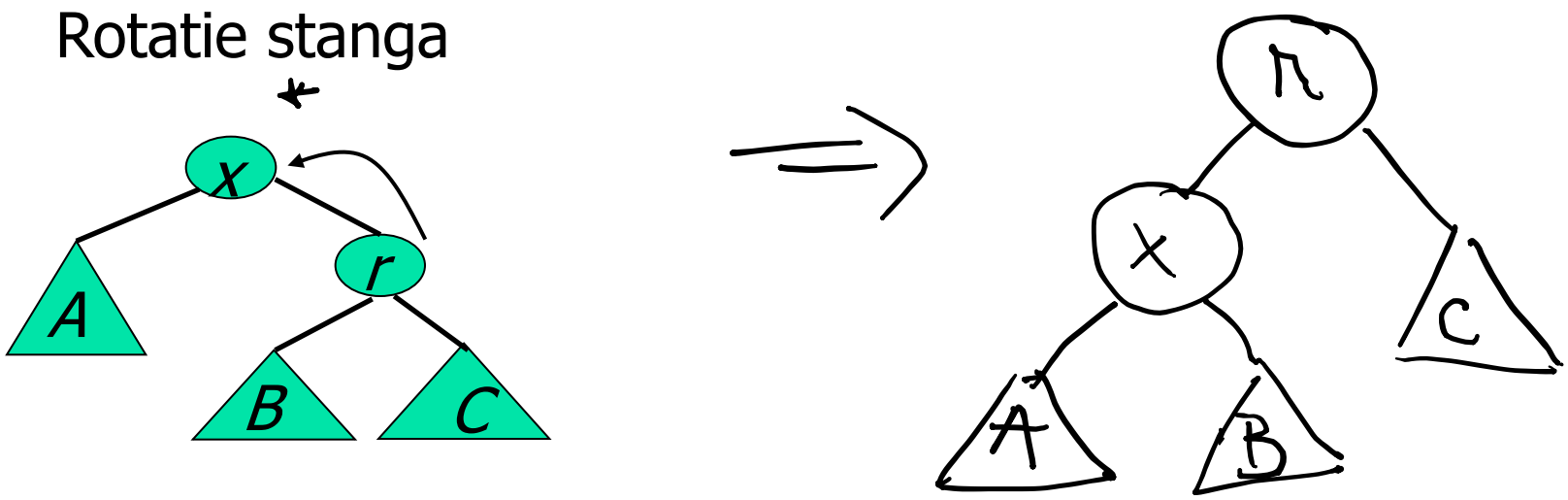
- se insereaza informatia in treap folosind regula de inserare de la arbori binari de cautare
- se genereaza o prioritate aleatoare pentru nodul inserat

- se actualizeaza structura arborelui pentru a se asigura conditia de heap raportata la prioritatile din nodurile arborelui: **rotatii stanga / dreapta**

Rotatii

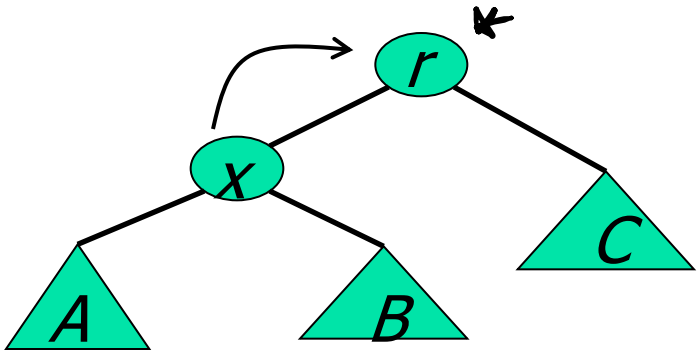


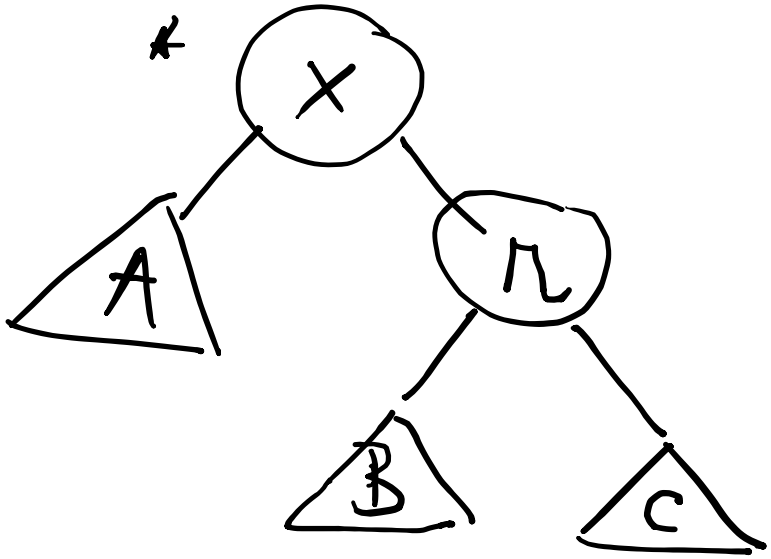
Rotatie stanga / dreapta



Rotatie stanga / dreapta

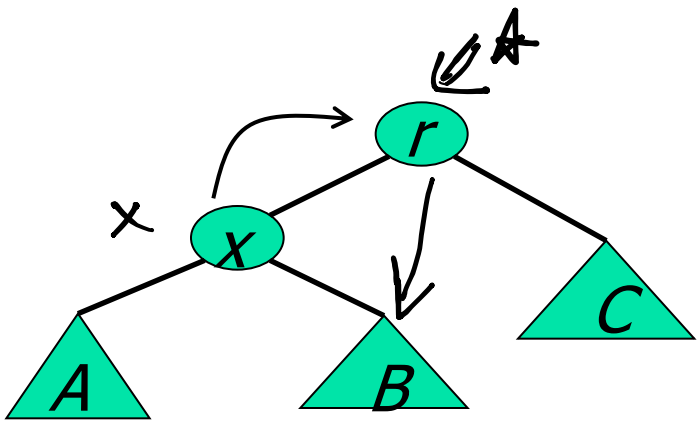
Rotatie dreapta



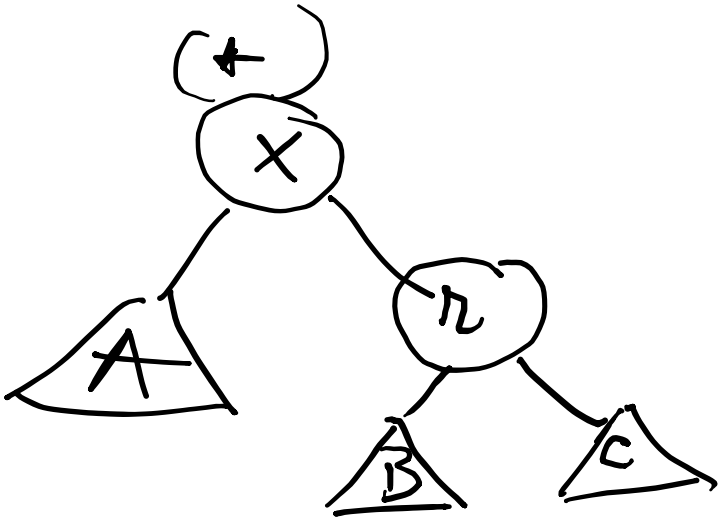


Rotatie stanga / dreapta

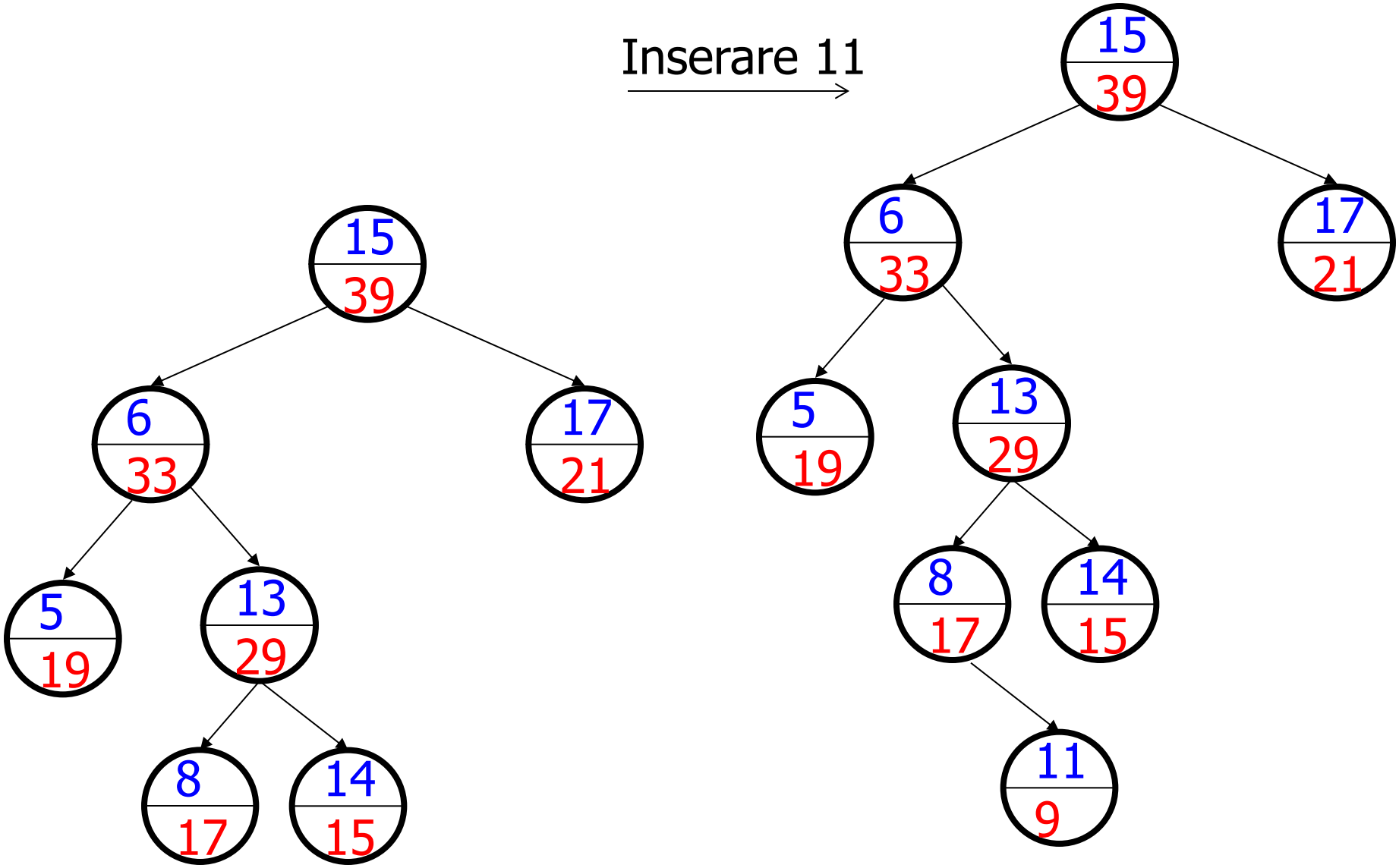
Rotatie dreapta



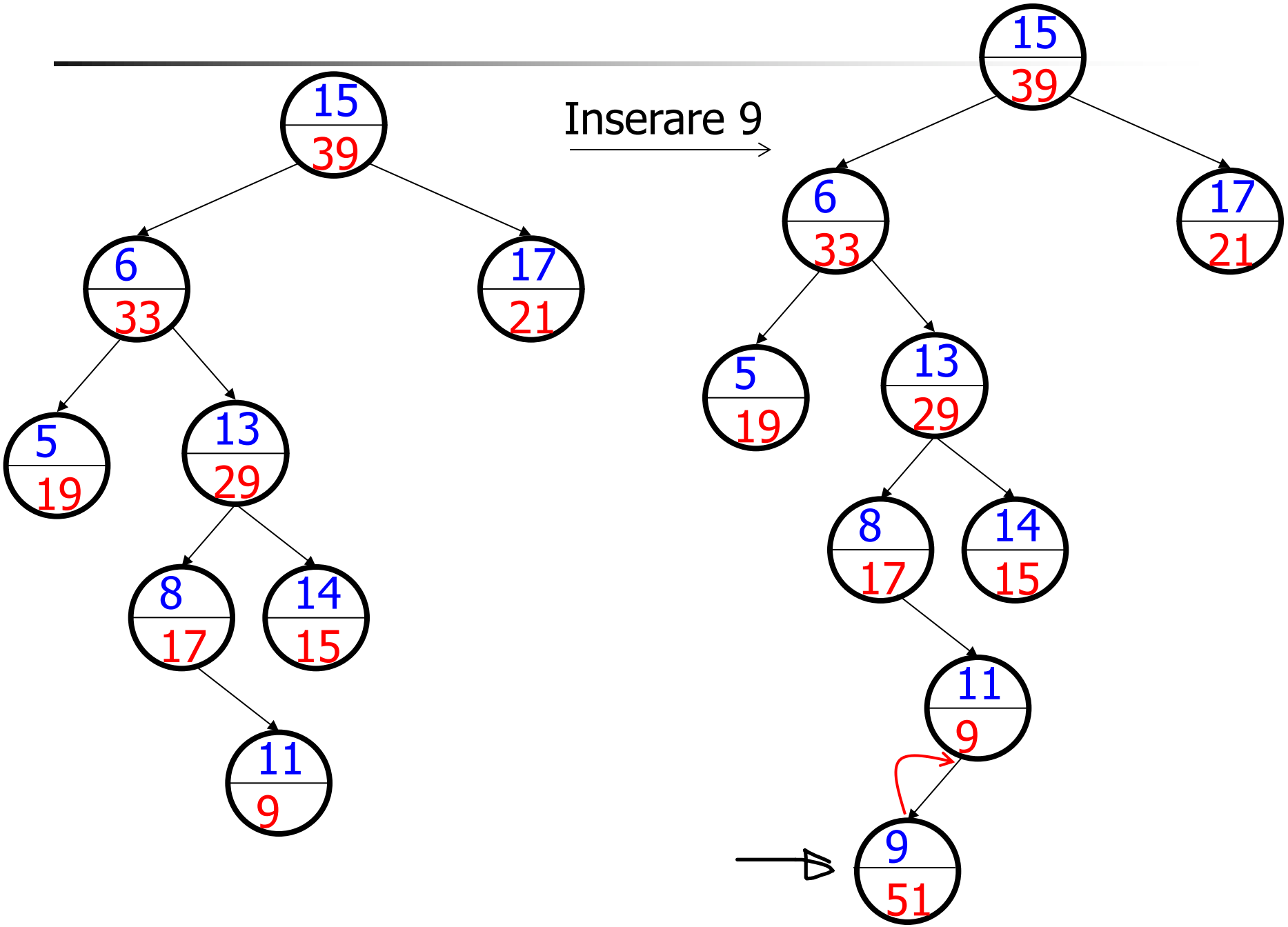

```
x = r->st;
r->st = x->dr;
x->dr = r;
r <- x;
```



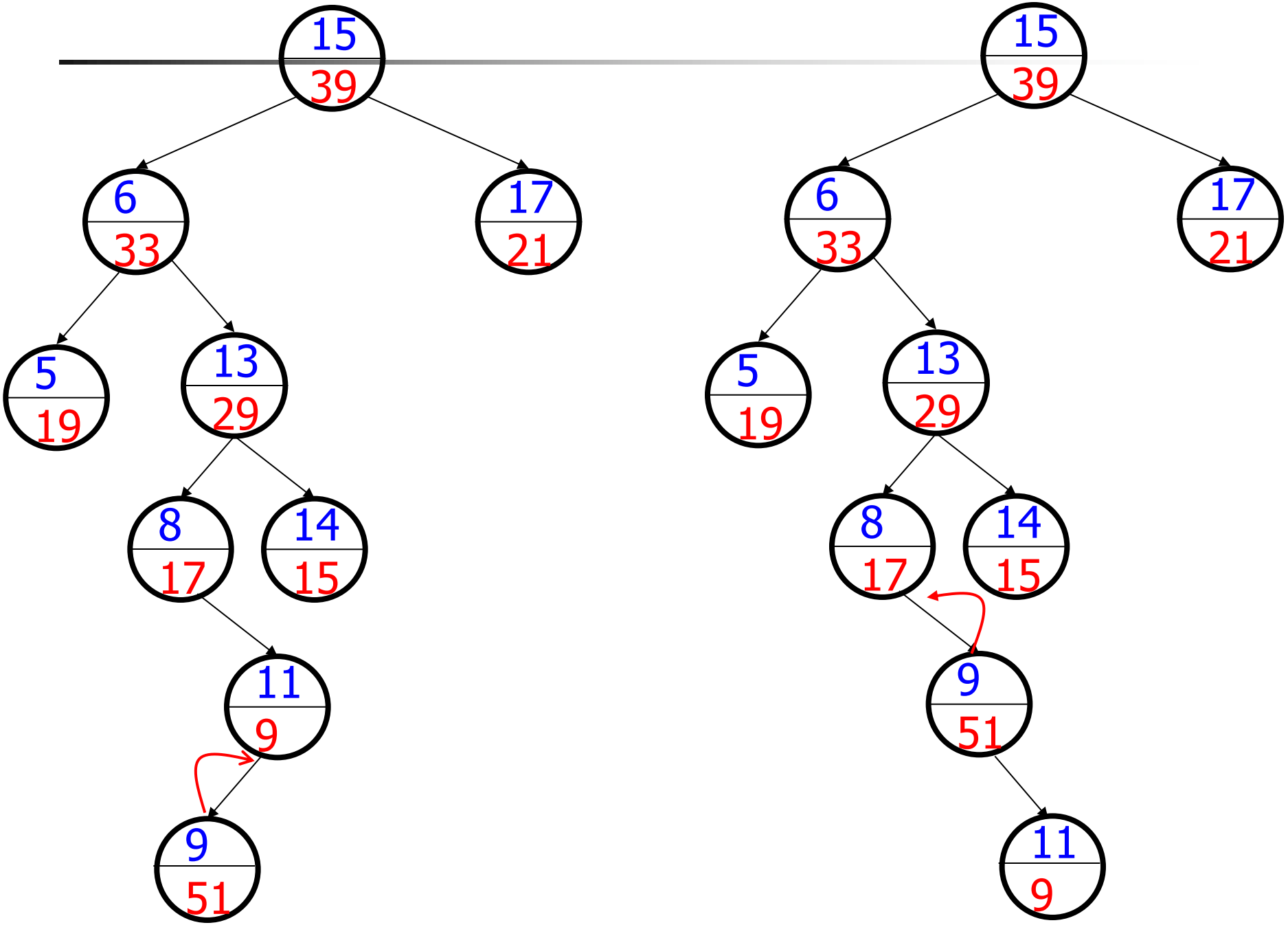
Exemplu inserare



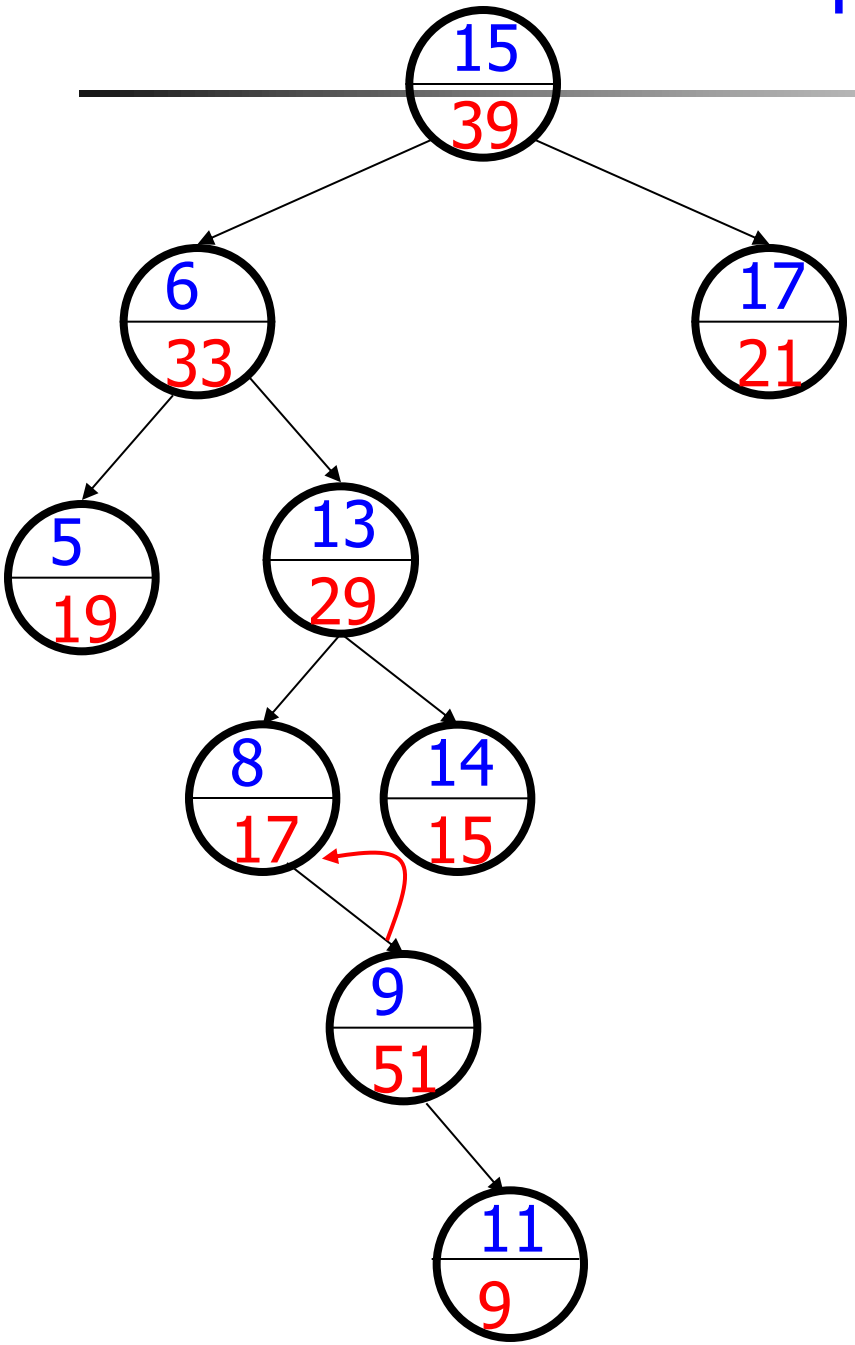
Exemplu inserare

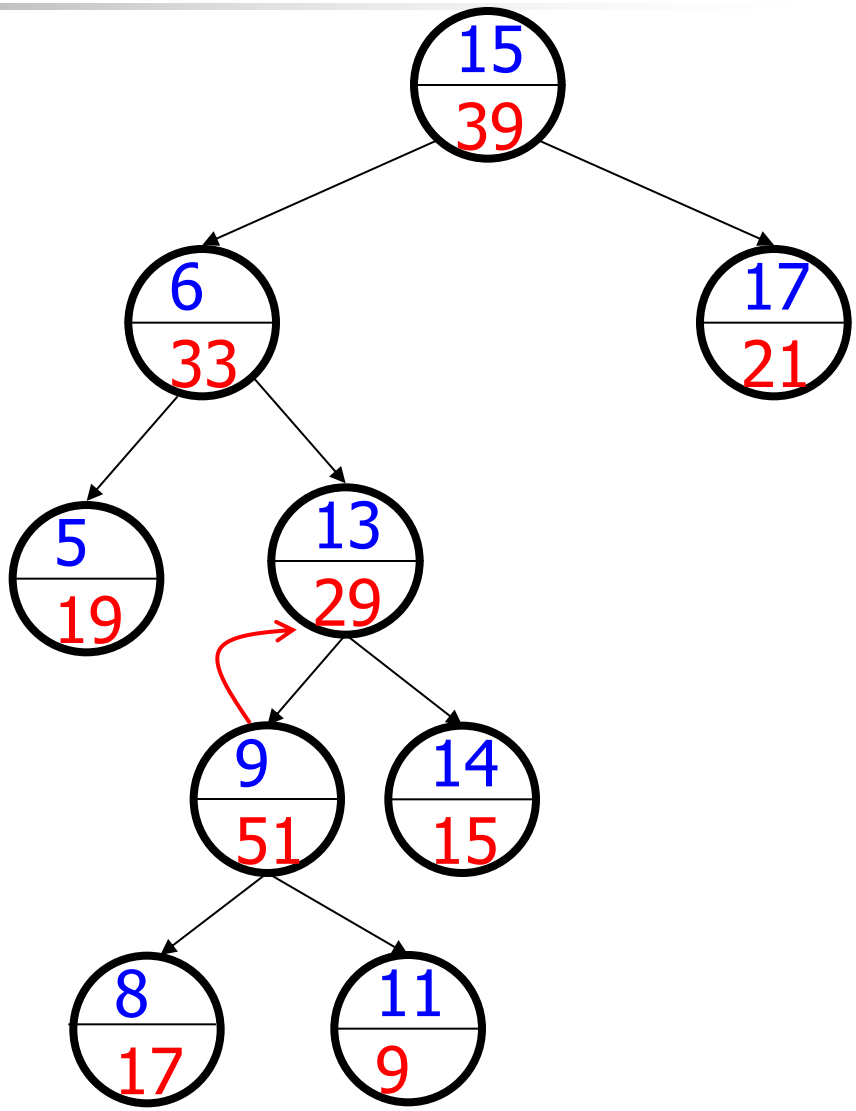


Exemplu inserare

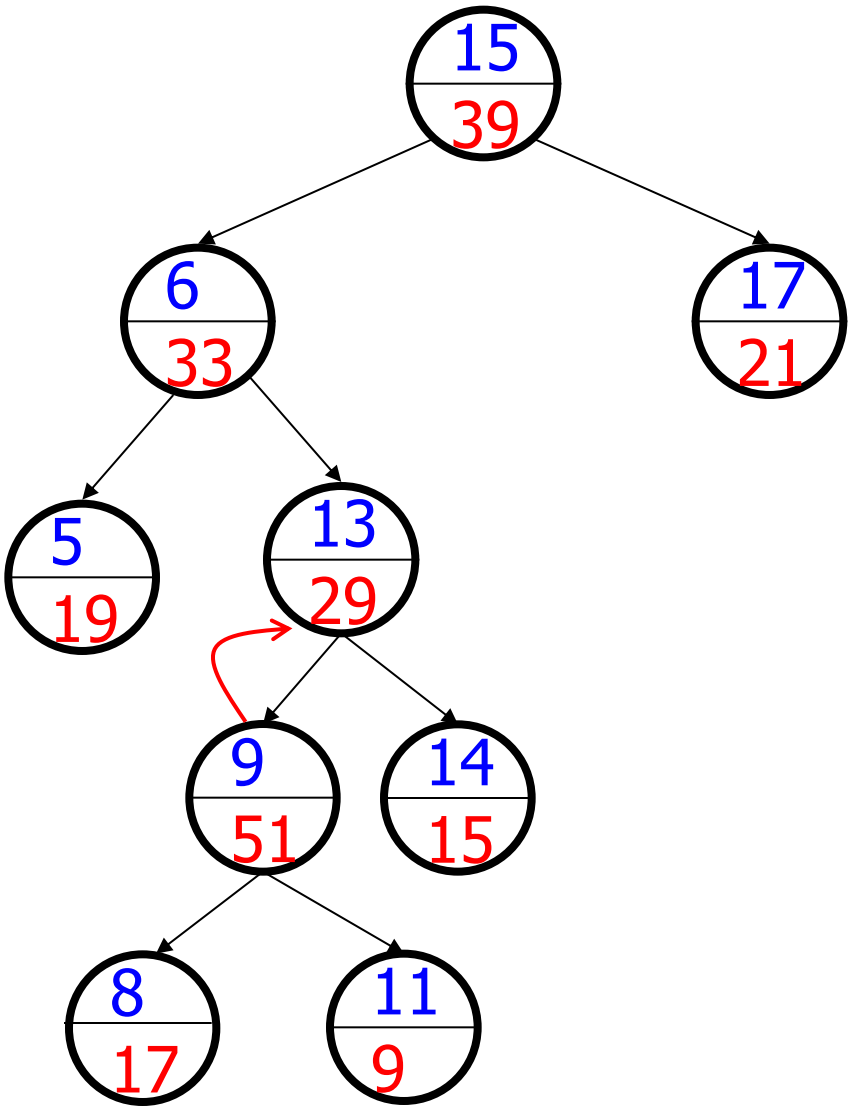


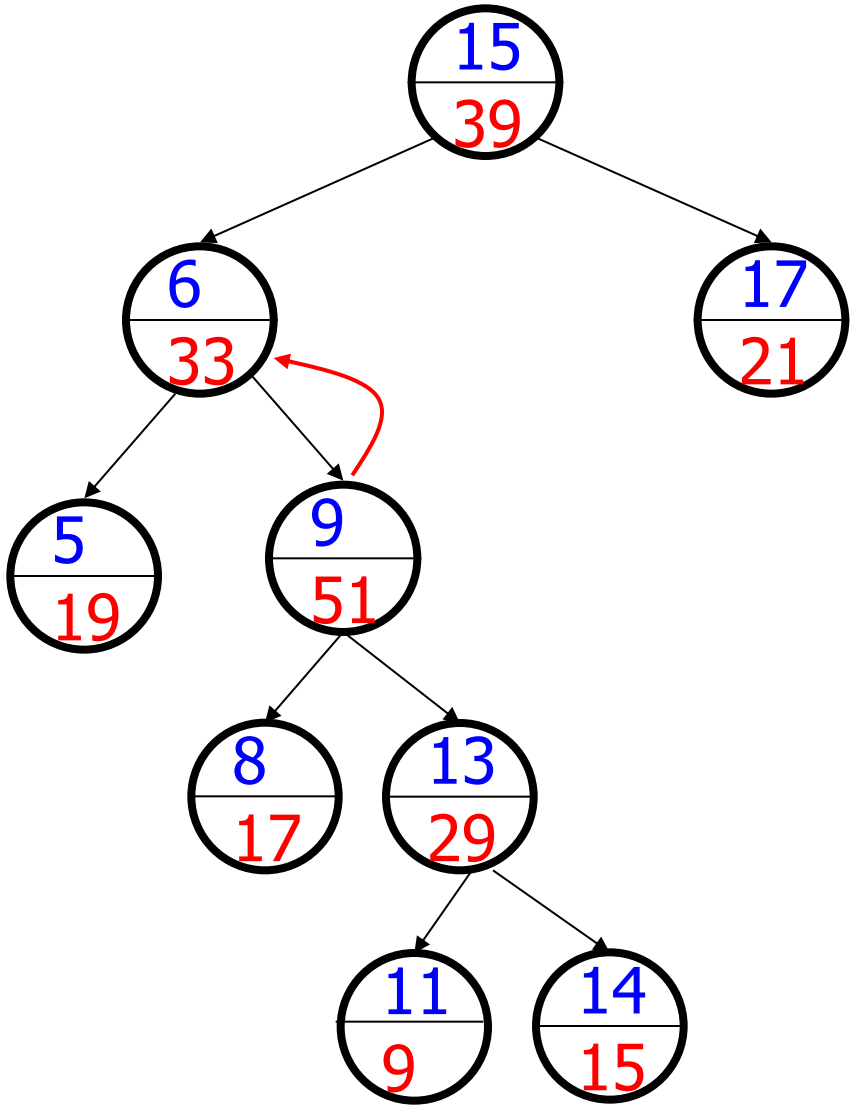
Exemplu inserare



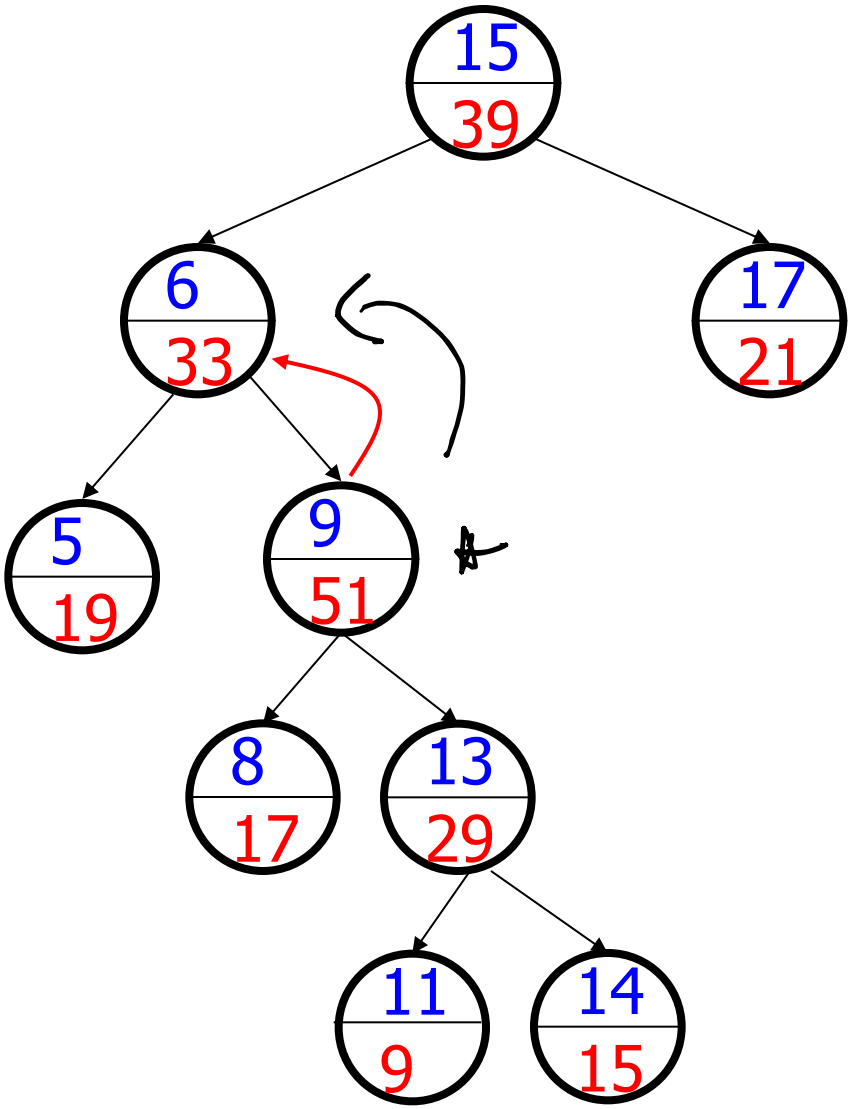


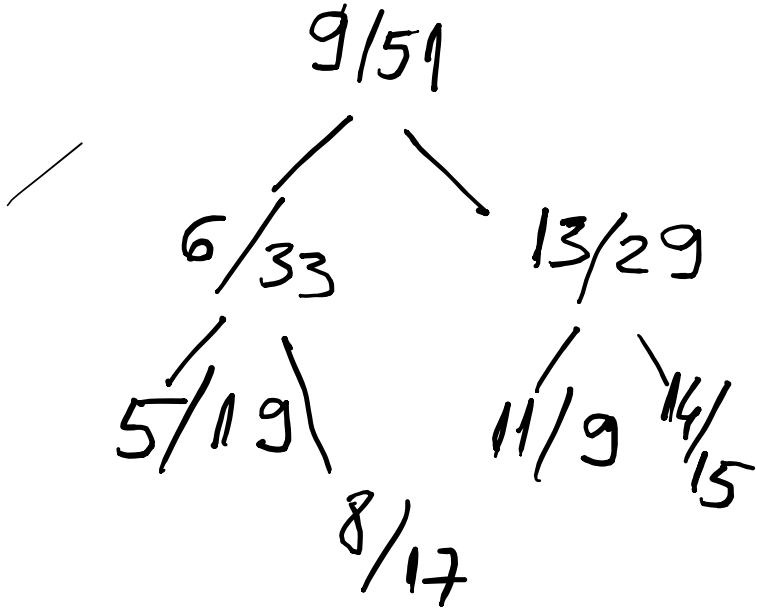
Exemplu inserare



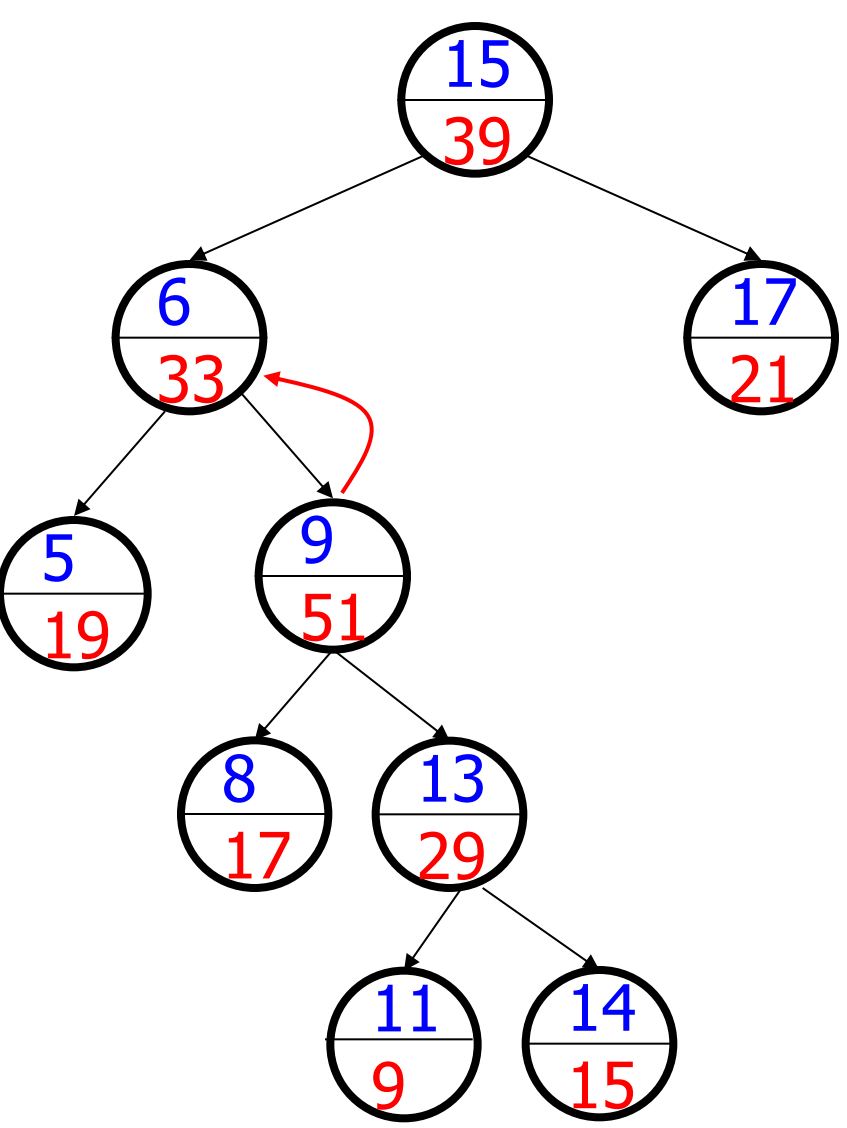


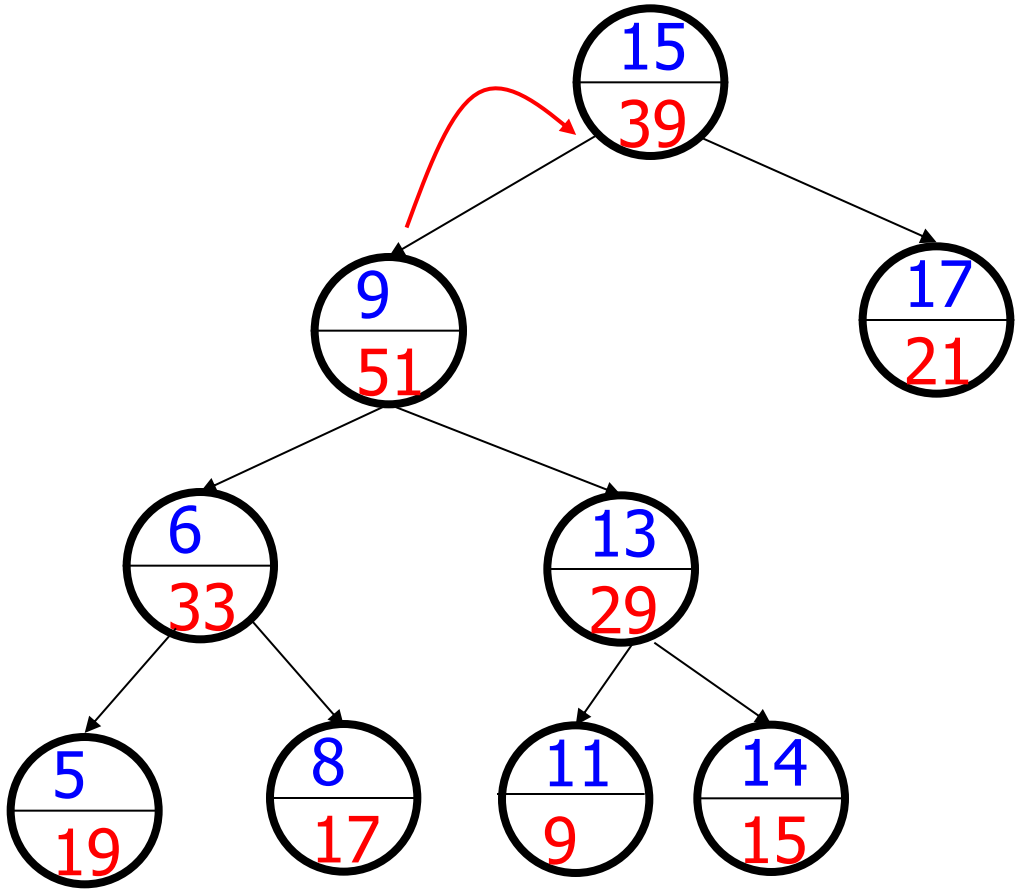
Exemplu inserare



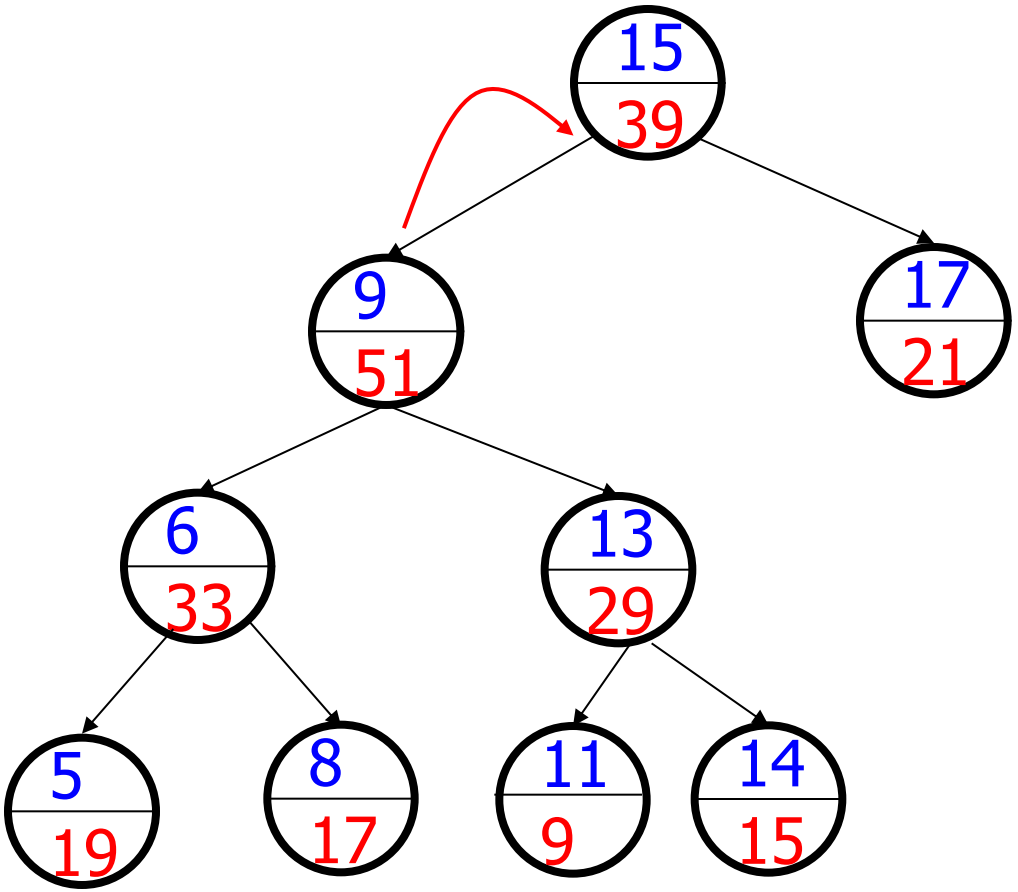


Exemplu inserare

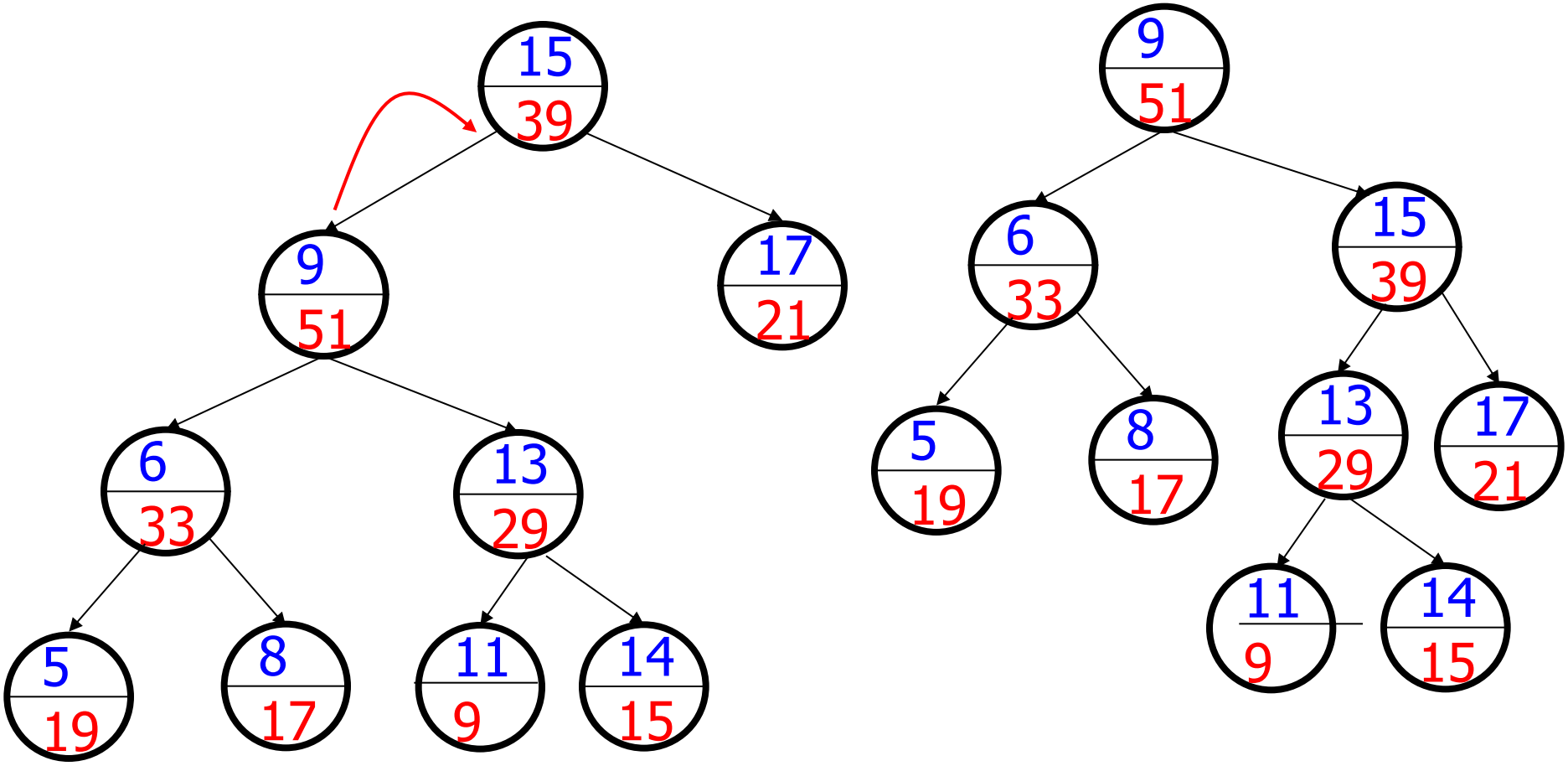




Exemplu inserare

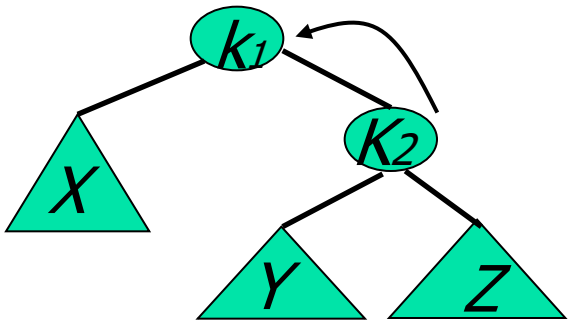


Exemplu inserare

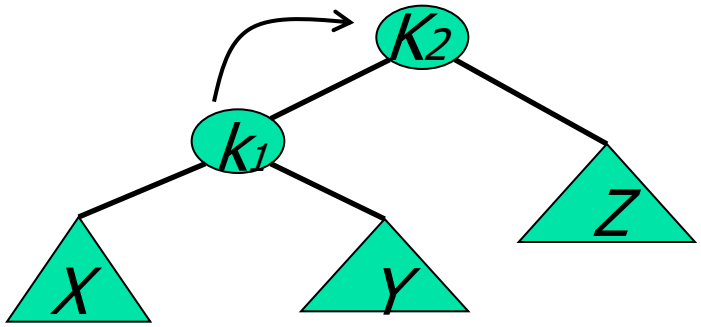


Rotatie stanga / dreapta

Rotatie stanga



Rotatie dreapta



Inserare in treap cheia x

- Genereaza prioritate pentru cheia x
- Insereaza x in treap – inserare frunza analog inserarii in arbore binar de cautare
- Reface proprietatea de heap prin rotatii stanga / dreapta

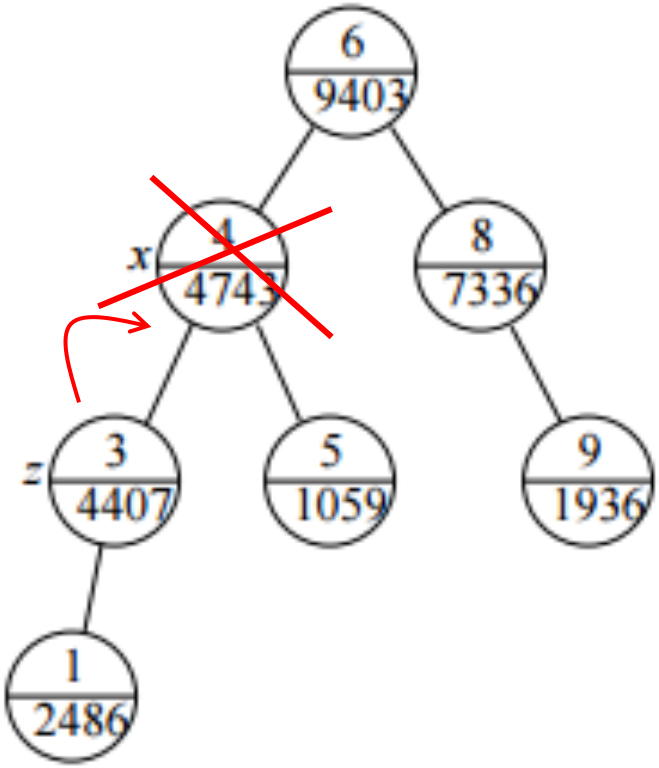
Inserare in treap

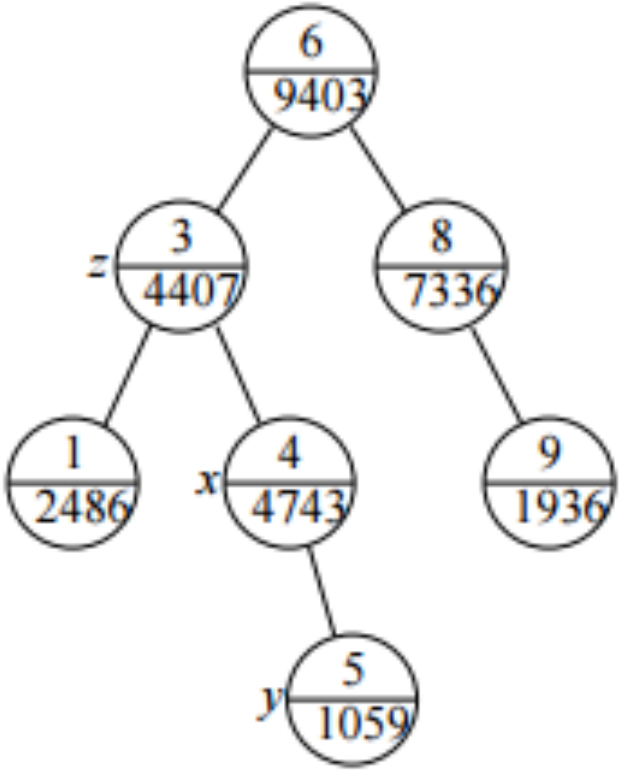
```
void insert(nod, cheie, prioritate)
{ if nod == null
        nod = creza nou nod pe baza de cheie si prioritate
  else
        if cheie < nod->info
                insert(nod->st, cheie, prioritate)
        else
                insert(nod->dr, cheie, prioritate)

  if nod->st->prioritate > nod->prioritate
        rotireDreapta(nod)
  else
        if nod->dr->prioritate > nod->prioritate
                rotireStanga(nod) }
```

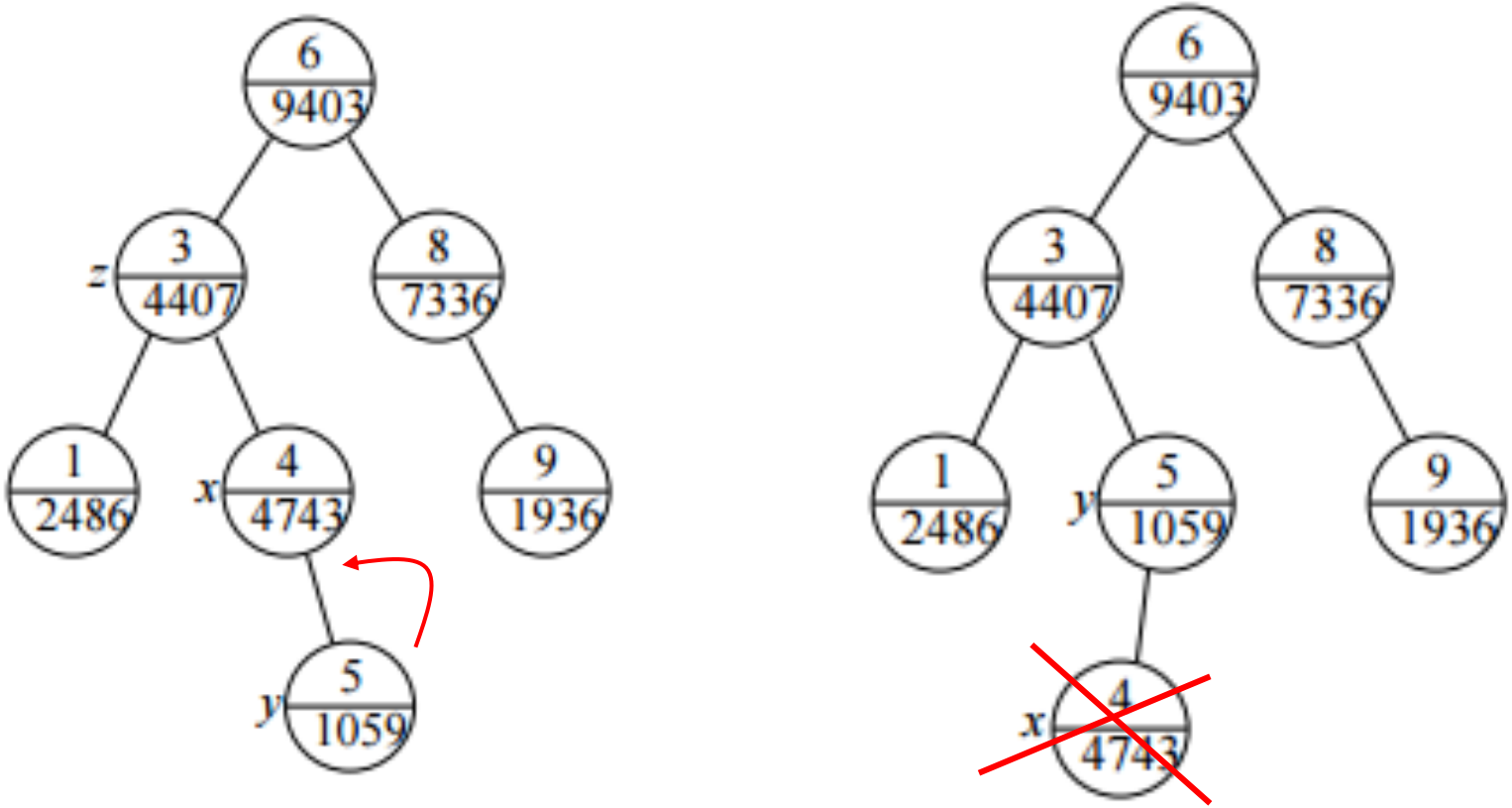
Eliminare

- Operatia inversa inserarii





Eliminare


6
9403
z
3
4407
8
7336
1
2486
x
4
4743
9
1936
y
5
1059
6
9403
3
4407
8
7336
1
2486
y
5
1059
9
1936
x
4
4743

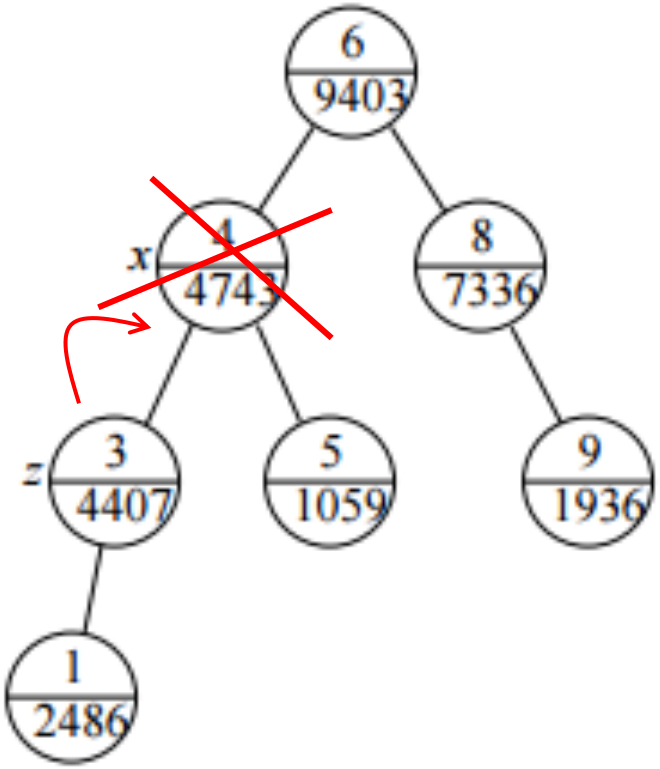
Eliminare

- Cauta *x* in arbore (analog cautare in arbore binar de cautare)

- Daca x este frunza atunci sterge nod
- Altfel propaga nodul cu informatia x pana ajunge o frunza (prin rotatii succesive)

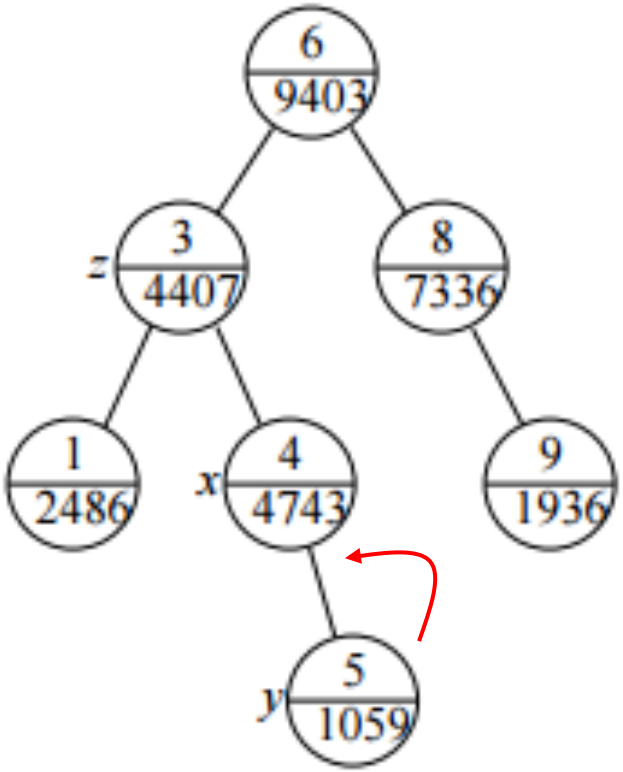
- Sterge frunza ce contine informatia x din arbore

Eliminare




```
void sterge(nod, x)
{ if nod == null return
  if x < nod->info
        sterge(nod->st, x)
  else if x > nod->info
        sterge(nod->dr, x)
       else if frunza(nod)
                sterge nod
            else
                if nod->st->prioritate > nod->dr->prioritate
                        rotireDreapta(nod);
                        sterge(nod, x);
                else
                        rotireStanga(nod);
                        sterge(nod, x) }
```

Eliminare




```
void sterge(nod, x)
{ if nod == null return
  if x < nod->info
        sterge(nod->st, x)
  else if x > nod->info
        sterge(nod->dr, x)
       else if frunza(nod)
                sterge nod
            else
                if nod->st->prioritate > nod->dr->prioritate
                        rotireDreapta(nod);
                        sterge(nod, x);
                else
                        rotireStanga(nod);
                        sterge(nod, x) }
```